

# STAR ESP32 Firmware Implementation Primer

ESP32-S3 Motor Control and Sensor Integration Guide

STAR Development Team

December 2025

## Contents

## 1 Project Overview

### 1.1 Purpose

The `star-esp32-firmware` project provides real-time motor control, sensor integration, and communication with the Raspberry Pi 5 gateway for the STAR robot platform. It runs on the ESP32-S3 microcontroller using FreeRTOS.

### 1.2 Current Implementation Status

| Component                 | Status      | Notes                                     |
|---------------------------|-------------|-------------------------------------------|
| STAR Framework Core       | Complete    | DIP architecture, interfaces              |
| Bus Abstraction Layer     | Complete    | I2C, SPI, UART, GPIO, 1-Wire, SMBus       |
| Sensor Drivers            | Complete    | HC-SR04, MPU6050, BNO055, PCA9685, BQ7850 |
| FreeRTOS Tasks            | Complete    | DHT22, Sensor, LED, Watchdog tasks        |
| Error Handling            | Complete    | Retry logic with exponential backoff      |
| Pin Validation            | Complete    | GPIO conflict detection                   |
| Motor Control (MCPWM)     | <b>TODO</b> | Center-aligned PWM for 4 motors           |
| SPI Peripheral (Pi5 comm) | <b>TODO</b> | Binary protocol for commands              |
| TCP Server                | <b>TODO</b> | Fallback communication channel            |
| PID Controller            | <b>TODO</b> | Velocity control loop at 1kHz             |
| Encoder Reading           | <b>TODO</b> | Quadrature decoder for odometry           |

Table 1: Implementation Status Summary

## 2 Architecture Overview

### 2.1 Dependency Inversion Principle (DIP)

The firmware follows SOLID principles with emphasis on Dependency Inversion:

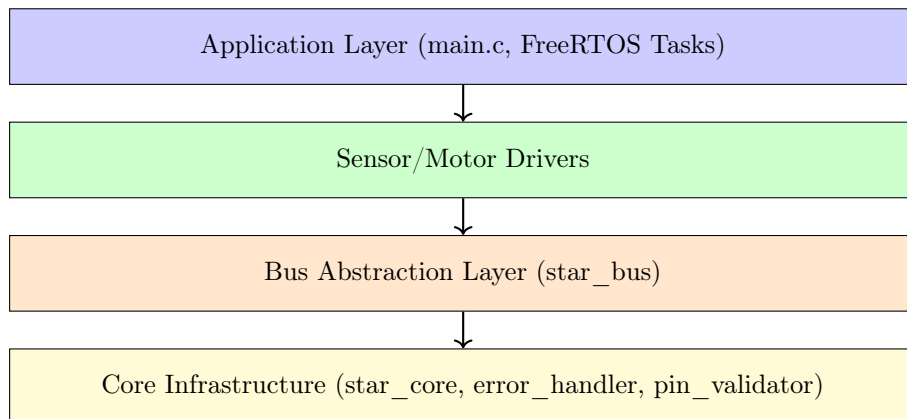


Figure 1: STAR Firmware Layered Architecture

### 2.2 Directory Structure

```

1 star-esp32-firmware/
2 |-- src/
3 |   |-- main.c                # Application entry point
4 |   |-- system_config.c/h     # Hardware configuration
5 |   |-- modules/
6 |       |-- shared_data.c/h   # Thread-safe inter-task comm
7 |       +-- led_controller.c/h # LED control logic
8 |   +-- tasks/
9 |       |-- dht22_task.c/h     # Temperature monitoring
10 |       |-- sensor_task.c/h   # HC-SR04 distance
11 |       |-- led_task.c/h      # RGB LED feedback
12 |       +-- watchdog_task.c/h # System health
13 |
14 |-- lib/
15 |   |-- star_core/            # Abstract interfaces
16 |   |-- star_error_handler/   # Error handling
17 |   |-- star_pin_validator/   # GPIO conflict detection
18 |   |-- star_bus/             # Unified bus abstraction
19 |   |-- star_sensor_*/        # Sensor drivers
20 |   +-- star_bms_bq7850/      # Battery management
21 |
22 |-- platformio.ini           # Build configuration
23 +-- sdkconfig.*               # ESP-IDF configs

```

## 3 Implementation Tasks

### 3.1 Phase 1: Motor Control Implementation

#### 3.1.1 Task 1.1: MCPWM Driver

Create `lib/star_motor/` with center-aligned PWM support:

1. Create `star_motor.h` with motor handle and configuration structs
2. Implement MCPWM initialization for 4 motors (2 MCPWM units, 2 operators each)
3. Configure center-aligned (up-down count) mode for reduced EMI
4. Implement dead-time insertion (prevent H-bridge shoot-through)
5. Add fault input handling for overcurrent protection

#### Key Parameters:

- PWM Frequency: 20 kHz (above audible range)
- Dead Time: 500ns (DRV8243 requirement)
- Duty Cycle Resolution: 0.1%

Listing 1: Motor Driver Interface

```

1  typedef struct {
2      mcpwm_unit_t unit;
3      mcpwm_timer_t timer;
4      gpio_num_t pwm_a;
5      gpio_num_t pwm_b;
6      gpio_num_t fault_pin;
7      float max_duty;
8  } star_motor_config_t;
9
10 esp_err_t star_motor_init(star_motor_handle_t* handle,
11                           const star_motor_config_t* config,
12                           star_error_interface_t* error_iface);
13 esp_err_t star_motor_set_velocity(star_motor_handle_t* handle,
14                                   float velocity_mps);
15 esp_err_t star_motor_brake(star_motor_handle_t* handle);
16 esp_err_t star_motor_coast(star_motor_handle_t* handle);

```

### 3.1.2 Task 1.2: Quadrature Encoder Driver

Create lib/star\_encoder/ for velocity feedback:

1. Use ESP32 PCNT (Pulse Counter) peripheral
2. Configure for quadrature decoding (4x resolution)
3. Implement velocity calculation from count delta
4. Add overflow handling for continuous operation

#### Encoder Specifications:

- Resolution: 341.2 counts/revolution
- Wheel diameter: 100mm
- Distance per count: 0.92mm

### 3.1.3 Task 1.3: PID Controller

Create lib/star\_pid/ for closed-loop velocity control:

Listing 2: PID Controller Interface

```

1  typedef struct {
2      float kp, ki, kd;           // Gains
3      float integral_limit;       // Anti-windup
4      float output_limit;         // Max PWM duty
5      float deadband;             // Ignore small errors
6  } star_pid_config_t;
7
8  void star_pid_init(star_pid_t* pid, const star_pid_config_t* config);
9  float star_pid_update(star_pid_t* pid, float setpoint, float measurement,
10                       float dt);
11 void star_pid_reset(star_pid_t* pid);

```

**Initial Tuning Values (adjust experimentally):**

- $K_p = 1.0$ ,  $K_i = 0.5$ ,  $K_d = 0.1$
- Loop rate: 1000 Hz
- Integral limit:  $\pm 100$

**3.2 Phase 2: Communication Implementation****3.2.1 Task 2.1: SPI Peripheral Driver**

Implement SPI peripheral mode for Pi5 communication:

1. Configure ESP32 as SPI peripheral on SPI2 (HSPI)
2. Implement binary command protocol
3. Add DMA for efficient transfers
4. Handle command parsing in ISR-safe manner

**SPI Pin Mapping:**

| Signal | Pi5 GPIO | ESP32 GPIO |
|--------|----------|------------|
| COPI   | GPIO11   | GPIO13     |
| CPO    | GPIO9    | GPIO12     |
| SCLK   | GPIO11   | GPIO14     |
| CS     | GPIO8    | GPIO15     |

**Command Protocol:**

Listing 3: SPI Command Protocol

```

1  // Command byte definitions
2  #define CMD_SET_VELOCITY    0x01  // Set wheel velocities
3  #define CMD_EMERGENCY_STOP  0x02  // Immediate stop
4  #define CMD_GET_ENCODERS    0x03  // Read encoder counts
5  #define CMD_SET_MOTOR       0x04  // Direct motor control
6  #define CMD_PING            0x05  // Connection test
7  #define CMD_GET_TELEMETRY    0x06  // Battery, temp, status
8
9  // Packet format: [CMD (1)] [LENGTH (1)] [DATA (N)] [CRC8 (1)]
10 typedef struct __attribute__((packed)) {
11     uint8_t command;
12     uint8_t length;
13     uint8_t data[32];
14     uint8_t crc8;
15 } spi_packet_t;
16
17 // Velocity command payload
18 typedef struct __attribute__((packed)) {
19     float left_velocity;  // m/s
20     float right_velocity; // m/s
21     uint32_t timestamp;   // ms since boot
22 } velocity_cmd_t;

```

### 3.2.2 Task 2.2: TCP Server (Fallback)

Implement TCP server as backup communication channel:

1. Create TCP server on port 5000
2. Accept JSON-formatted commands
3. Integrate with WiFi station mode (connect to Pi5 AP)
4. Implement connection timeout and reconnection

Listing 4: TCP Command Format (JSON)

```

1 // Velocity command
2 {"command": "velocity", "params": {"left": 0.5, "right": 0.5}}
3
4 // Emergency stop
5 {"command": "emergency_stop"}
6
7 // Response format
8 {"success": true, "message": "OK", "data": {...}}
```

## 3.3 Phase 3: Integration

### 3.3.1 Task 3.1: Motor Control Task

Create `src/tasks/motor_task.c`:

1. Run PID loop at 1000 Hz (1ms period)
2. Read encoder velocity feedback
3. Compute PID output and apply to MCPWM
4. Publish odometry to shared data
5. Handle emergency stop interrupt

Listing 5: Motor Control Task Structure

```

1 void motor_control_task(void* pvParameters) {
2     TickType_t last_wake = xTaskGetTickCount();
3     const TickType_t period = pdMS_TO_TICKS(1); // 1kHz
4
5     while (1) {
6         // 1. Read velocity setpoints from queue
7         velocity_cmd_t cmd;
8         if (xQueuePeek(velocity_queue, &cmd, 0) == pdTRUE) {
9             target_left = cmd.left_velocity;
10            target_right = cmd.right_velocity;
11        }
12
13        // 2. Read encoder velocities
```

```

14     float vel_left = star_encoder_get_velocity(&enc_left);
15     float vel_right = star_encoder_get_velocity(&enc_right);
16
17     // 3. Update PID controllers
18     float duty_left = star_pid_update(&pid_left, target_left,
19                                     vel_left, 0.001f);
20     float duty_right = star_pid_update(&pid_right, target_right,
21                                     vel_right, 0.001f);
22
23     // 4. Apply to motors
24     star_motor_set_duty(&motor_left, duty_left);
25     star_motor_set_duty(&motor_right, duty_right);
26
27     // 5. Update odometry
28     update_odometry(vel_left, vel_right, 0.001f);
29
30     vTaskDelayUntil(&last_wake, period);
31 }
32 }

```

### 3.3.2 Task 3.2: Communication Task

Create `src/tasks/comm_task.c`:

1. Poll SPI peripheral for incoming commands
2. Parse and validate packets
3. Dispatch to appropriate handlers
4. Send responses/telemetry
5. Fallback to TCP if SPI fails

## 4 Testing Plan

### 4.1 Unit Tests

1. **PID Controller**: Test with simulated plant, verify convergence
2. **Encoder Driver**: Test count accuracy, overflow handling
3. **Motor Driver**: Test PWM generation, dead-time
4. **SPI Protocol**: Test packet parsing, CRC validation

### 4.2 Integration Tests

1. **Motor + Encoder**: Closed-loop step response test
2. **SPI + Motor**: Command-to-motion latency measurement
3. **Full System**: Velocity tracking accuracy at various speeds

### 4.3 Acceptance Criteria

| Metric                  | Target               |
|-------------------------|----------------------|
| PID loop rate           | 1000 Hz $\pm$ 1%     |
| Velocity tracking error | < 5% at steady state |
| Command latency (SPI)   | < 1ms                |
| Emergency stop time     | < 10ms               |

## 5 Build and Flash Instructions

### 5.1 Prerequisites

```

1 # Install PlatformIO
2 pip install platformio
3
4 # Install ESP-IDF (handled by PlatformIO)
5 pio pkg install

```

### 5.2 Build Commands

```

1 # Build for ESP32-S3
2 pio run -e esp32s3
3
4 # Build for ESP32-WROOM (dev board)
5 pio run -e esp32_wroom
6
7 # Upload firmware
8 pio run -e esp32s3 --target upload
9
10 # Monitor serial output
11 pio device monitor -b 115200

```

### 5.3 Configuration

Edit `sdkconfig.esp32s3` for production settings:

- Enable brownout detection
- Set CPU frequency to 240 MHz
- Configure flash size (16 MB)
- Enable PSRAM if needed

## 6 References

- ESP-IDF MCPWM Documentation: <https://docs.espressif.com/projects/esp-idf/en/latest/esp32s3/api-reference/peripherals/mcpwm.html>



- ESP-IDF SPI Peripheral: [https://docs.espressif.com/projects/esp-idf/en/latest/esp32s3/api-reference/peripherals/spi\\_slave.html](https://docs.espressif.com/projects/esp-idf/en/latest/esp32s3/api-reference/peripherals/spi_slave.html)
- FreeRTOS Task Notifications: <https://www.freertos.org/RTOS-task-notifications.html>
- DRV8243 Datasheet: TI SLVSFN4
- STAR FTP Section 16: Embedded Implementation
- STAR Software Architecture Primer: Section 12 (ESP32 Firmware)